Ficha de Trabalho: Atributos de Processos e Contexto de Execução em POSIX

Objetivos de Aprendizagem:

- Compreender o conceito de processo, identificadores de processo e atributos do utilizador/grupo em sistemas operativos POSIX (Linux/UNIX).
- Utilizar programaticamente as chamadas de sistema (*System Calls*) e funções da biblioteca padrão POSIX: getpid(), getppid(), getuid(), getgid(), getenv() e getcwd().
- Manipular o ambiente e o diretório de trabalho do processo via código na linguagem C.

Resumo das Chamadas de Sistema POSIX

Função / System Call	Ficheiro de Cabeçalho (Header)	Descrição Resumida
getpid()	<unistd.h>, <sys/types.h>	Retorna o identificador do processo atual (PID).
getppid()	<unistd.h>, <sys/types.h>	Retorna o identificador do processo pai (PPID).
getuid()	<unistd.h>, <sys/types.h>	Retorna o ID do utilizador real (UID) do processo.
getgid()	<unistd.h>, <sys/types.h>	Retorna o ID do grupo real (GID) do processo.
getenv()	<stdlib.h>	Procura e retorna o valor de uma variável de ambiente.
getcwd()	<unistd.h>	Copia o caminho absoluto

Função / System Call	Ficheiro de Cabeçalho (Header)	Descrição Resumida
		do diretório atual de trabalho para um buffer.

Exercício 1: Identificadores de Processos (PID e PPID)

Os processos em sistemas POSIX são identificados univocamente através de números inteiros positivos chamados *Process IDs* (PID). Cada processo possui também um processo pai (*Parent Process*), identificado pelo PPID.

1. **Análise de Funções POSIX:**
   Consulte a documentação do sistema (man 2 getpid / man 2 getppid) e descreva:
   - O objetivo das funções getpid() e getppid().
   - Os tipos de dados que retornam e os parâmetros que aceitam.
2. **Implementação Básica:**
   Considere o seguinte código em C (print_pid.c):

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {
    printf("PID do processo atual: %d\n", (int) getpid());
    printf("PID do processo pai (PPID): %d\n", (int) getppid());
    return 0;
}
```

   - Compile e execute o programa múltiplas vezes a partir do terminal.
   - O PID impresso é sempre o mesmo a cada execução? E o PPID? Justifique o comportamento observado.
3. **Análise de Hierarquia de Processos:**
   - Qual é o processo representado pelo PPID que é impresso quando executa o programa no terminal?
   - Como é que o sistema operativo garante a unicidade do PID durante a execução em simultâneo de múltiplos programas?

Exercício 2: Credenciais do Processo (UID e GID)

Em sistemas POSIX, a permissão e o acesso a recursos (ficheiros, dispositivos, sockets, etc.) são definidos pelas credenciais do utilizador (*User ID* — UID) e grupo (*Group ID* — GID) associados ao processo.

1. **Análise de Funções POSIX:**
   Consulte a documentação (man 2 getuid / man 2 getgid) e descreva:
   - O objetivo das funções getuid() e getgid().
   - A diferença conceitual entre o utilizador real (*Real UID*) e o utilizador efetivo (*Effective UID*).
2. **Exercício Prático:**
   - Escreva um programa em C que obtenha e apresente o UID e o GID do processo em execução.
   - Executando o programa sob diferentes utilizadores do sistema, que alterações são observadas nas saídas das funções getuid() e getgid()?

Exercício 3: Variáveis de Ambiente e Diretório de Trabalho (getenv e getcwd)

Cada processo herda e mantém o seu próprio ambiente (conjunto de pares chave-valor) e um diretório de trabalho atual (*Current Working Directory*).

1. **Análise de Funções POSIX:**
   Consulte a documentação (man 3 getenv e man 3 getcwd / man 2 getcwd) e responda:
   - Qual o objetivo de getenv() e getcwd()?
   - Quais são os parâmetros necessários para getcwd() e como deve ser gerido o buffer de memória onde o caminho é armazenado?
   - O que acontece se o buffer passado a getcwd() for menor do que o caminho do diretório atual?
2. **Desenvolvimento de Programa Integrado:**
   Crie um programa em C que extraia e formate num único relatório textual no terminal todas as informações sobre o processo corrente. O programa deve exibir:
   - **UID** (User ID)
   - **GID** (Group ID)
   - **PID** (Process ID)
   - **PPID** (Parent Process ID)
   - O valor das variáveis de ambiente **PATH** e **HOME**
   - O caminho absoluto do **Diretório Atual de Trabalho** (CWD)

   Exemplo de Saída Esperada:
   ```
   === Informação do Processo ===
   UID  = 1000
   GID  = 1000
   ```

```
PID  = 14205
PPID = 8120
PATH = /usr/local/sbin:/usr/local/bin:/usr/bin:/bin
HOME = /home/utilizador
CWD  = /home/utilizador/projetos/so_ficha2
```

3. **Desafio / Modificação do Ambiente:**
   - Pesquise sobre as funções setenv() e chdir().
   - Adicione ao seu programa instruções para alterar o diretório atual de trabalho (chdir) e verificar se a chamada subsequente a getcwd() reflete essa mudança dentro da execução do programa.

Bibliografias e Recursos de Consulta

1. ---

   Kernighan, B. W., & Ritchie, D. M. — *The C Programming Language* (2nd Ed.). Prentice Hall.
2. **Kerrisk, M.** — *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.
3. **IEEE Std 1003.1 (POSIX.1)** — *Base Specifications / System Interfaces*.